

You are a code translator. Please translate the following correct Python code into Rust, ensuring the functionality and correctness. Keep the structure idiomatic for Rust and check the twice before you provide the code. And make sure the time complexity is the possible least. Python Code:

python

折叠 运行 复制

```
from typing import List

class Solution:
    class Robot:
        def __init__(self, position: int, health: int, direction: str, index: int):
            self.position = position
            self.health = health
            self.direction = direction
            self.index = index

    def survivedRobotsHealths(self, positions: List[int], healths: List[int], directions: List[str], n: int) -> List[int]:
        n = len(positions)
        vals = [self.Robot(positions[i], healths[i], directions[i], i) for i in range(n)]
        vals.sort(key=lambda x: x.position)
        stack = []
        for robot in vals:
            if robot.direction == 'R':
                stack.append(robot)
                continue
            gone = False
            while stack and stack[-1].health <= robot.health and stack[-1].direction == 'L':
                if stack[-1].health == robot.health:
                    stack.pop()
                    gone = True
                    break
                robot.health -= 1
                stack.pop()
            if not gone and stack and stack[-1].direction == 'R' and stack[-1].health < robot.health:
                stack[-1].health -= 1
                gone = True
            if not gone:
                stack.append(robot)
        stack.sort(key=lambda x: x.index)
        return [robot.health for robot in stack]
```

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

Rust Code:

rust

🔗 折叠复制

```
#[derive(Debug)]
struct Robot {
    position: i32,
    health: i32,
    direction: char,
    index: usize,
}

impl Solution {
    pub fn survived_robots_healths(positions: Vec<i32>, healths: Vec<i32>, directions: String) -
        let n = positions.len();
        let mut vals: Vec<Robot> = (0..n)
            .map(|i| Robot {
                position: positions[i],
                health: healths[i],
                direction: directions.chars().nth(i).unwrap(),
                index: i,
            })
            .collect();
        vals.sort_by_key(|x| x.position);
        let mut stack: Vec<Robot> = Vec::new();
        for robot in vals {
            if robot.direction == 'R' {
                stack.push(robot);
                continue;
            }
            let mut gone = false;
            while let Some(last) = stack.last_mut() {
                if last.direction != 'R' || last.health > robot.health {
                    break;
                }
                if last.health == robot.health {
                    stack.pop();
                    gone = true;
                }
            }
        }
        stack
```

```

        break;
    }
    robot.health -= 1;
    stack.pop();
}
if !gone {
    if let Some(last) = stack.last_mut() {
        if last.direction == 'R' && last.health > robot.health {
            last.health -= 1;
            gone = true;
        }
    }
}
if !gone {
    stack.push(robot);
}
}
stack.sort_by_key(|x| x.index);
stack.into_iter().map(|x| x.health).collect()
}
}

```

Line 37: Char 17: error: cannot assign to `robot.health`, as `robot` is not declared as mutable (solution.rs) | 37 | robot.health -= 1; | ^^^^^^^^^^^^^^^^^^^^^ cannot assign | help: consider changing this to be mutable | 22 | for mut robot in vals { |
 +++ For more information about this error, try `rustc --explain E0594`. error: could not compile `prog` (bin "prog") due to 1 previous error

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

Rust Code:

rust

◇ 折叠复制

```

#[derive(Debug)]
struct Robot {
    position: i32,
    health: i32,
    direction: char
}

```

```

        direction: char,
        index: usize,
    }
}

impl Solution {
    pub fn survived_robots_healths(positions: Vec<i32>, healths: Vec<i32>, directions: String) -
        let n = positions.len();
        let mut vals: Vec<Robot> = (0..n)
            .map(|i| Robot {
                position: positions[i],
                health: healths[i],
                direction: directions.chars().nth(i).unwrap(),
                index: i,
            })
            .collect();
        vals.sort_by_key(|x| x.position);
        let mut stack: Vec<Robot> = Vec::new();
        for mut robot in vals {
            if robot.direction == 'R' {
                stack.push(robot);
                continue;
            }
            let mut gone = false;
            while let Some(last) = stack.last_mut() {
                if last.direction != 'R' || last.health > robot.health {
                    break;
                }
                if last.health == robot.health {
                    stack.pop();
                    gone = true;
                    break;
                }
            }
            robot.health -= 1;
            stack.pop();
        }
        if !gone {
            if let Some(last) = stack.last_mut() {
                if last.direction == 'R' && last.health > robot.health {
                    last.health -= 1;
                    gone = true;
                }
            }
        }
    }
}

```

```

    }

    }

    if !gone {
        stack.push(robot);
    }
}

stack.sort_by_key(|x| x.index);
stack.into_iter().map(|x| x.health).collect()
}
}

```

The given Rust implementation of `survived_robots_healths` has a time complexity that exceeds the time limit on LeetCode due to multiple nested loops and frequent sorting of the robot list. Specifically:

1. The code uses `sort_by_key` multiple times, which is an $O(n \log n)$ operation, both initially and after each battle.
2. The battle simulation involves a while loop that interacts with a stack, which also adds unnecessary time complexity.

Please modify the code to optimize the logic by:

- Avoiding re-sorting the robot list after every battle.
- Reducing the complexity of the robot interactions by simulating the battles in a more efficient manner.
- Reconsidering the use of the stack structure and optimizing the conditions under which robots "survive."

Focus on reducing time complexity to $O(n)$, if possible, using more efficient data structures or algorithms to simulate the robot battles and health reductions.

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

Rust Code:

rust

🔼 折叠复制

```

#[derive(Debug)]
struct Robot {
    position: i32,
    health: i32,
    direction: char,
}

```

```

        index: usize,
    }

}

impl Solution {
    pub fn survived_robots_healths(positions: Vec<i32>, healths: Vec<i32>, directions: String) -
        let n = positions.len();
        let mut robots: Vec<Robot> = (0..n)
            .map(|i| Robot {
                position: positions[i],
                health: healths[i],
                direction: directions.chars().nth(i).unwrap(),
                index: i,
            })
            .collect();
        robots.sort_by_key(|x| x.position);
        let mut stack: Vec<Robot> = Vec::new();
        for robot in robots {
            if robot.direction == 'R' {
                stack.push(robot);
                continue;
            }
            let mut current_robot = robot;
            while let Some(mut last) = stack.pop() {
                if last.direction != 'R' {
                    stack.push(last);
                    break;
                }
                if last.health == current_robot.health {
                    current_robot.health = 0;
                    break;
                } else if last.health > current_robot.health {
                    last.health -= 1;
                    stack.push(last);
                    current_robot.health = 0;
                    break;
                } else {
                    current_robot.health -= 1;
                }
            }
            if current_robot.health > 0 {
                stack.push(current_robot);
            }
        }
        stack.reverse();
        stack.iter().map(|robot| robot.health).collect()
    }
}

```

```

        stack.push(current_robot);
    }
}

stack.sort_by_key(|x| x.index);
stack.into_iter().map(|x| x.health).collect()
}
}

```

Modify the code according to the following suggestion: Convert the direction string to `Vec<char>` ($O(n)$) in advance

Replace all `directions.chars().nth(i)` calls.

Avoid unnecessary sorting

Sort by position only once at the beginning; At the end, restore the order by index and save the result with `Vec<Option<i32>>`.

Avoid struct copying and try to use reference or index operations as much as possible

Store the stack in the index instead of the entire Robot, thereby reducing data movement.

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

Rust Code:

```

rust ◇ 折叠复制

#[derive(Debug)]
struct Robot {
    position: i32,
    health: i32,
    direction: char,
    index: usize,
}

impl Solution {
    pub fn survived_robots_healths(positions: Vec<i32>, healths: Vec<i32>, directions: String) -
        let n = positions.len();
        let directions: Vec<char> = directions.chars().collect();
        let mut robots: Vec<Robot> = (0..n)
            .map(|i| Robot {

```

```

        position: positions[i],
        health: healths[i],
        direction: directions[i],
        index: i,
    })
    .collect();
robots.sort_by_key(|x| x.position);
let mut stack: Vec<usize> = Vec::new();
let mut result: Vec<Option<i32>> = vec![None; n];
for i in 0..n {
    if robots[i].direction == 'R' {
        stack.push(i);
        continue;
    }
    let mut current_robot = &mut robots[i];
    while let Some(&last_idx) = stack.last() {
        let last_robot = &mut robots[last_idx];
        if last_robot.direction != 'R' {
            break;
        }
        if last_robot.health == current_robot.health {
            result[last_robot.index] = None;
            result[current_robot.index] = None;
            stack.pop();
            current_robot.health = 0;
            break;
        } else if last_robot.health > current_robot.health {
            last_robot.health -= 1;
            result[current_robot.index] = None;
            current_robot.health = 0;
            break;
        } else {
            result[last_robot.index] = None;
            stack.pop();
            current_robot.health -= 1;
        }
    }
    if current_robot.health > 0 {
        result[current_robot.index] = Some(current_robot.health);
    }
}

```

```

    }
    for idx in stack {
        result[robots[idx].index] = Some(robots[idx].health);
    }
    result.into_iter().filter_map(|x| x).collect()
}
}

```

In this line:

```
let last_robot = &mut robots[last_idx]; let current_robot = &mut robots[i];
```

You created two mutable references at the same time, `&mut robots[...]` Pointing to the same `Vec` is illegal in Rust's borrowing rules (even if the two indexes are different, Rust cannot statically guarantee non-overlap at compile time).

Furthermore, `stack.last()` only returns immutable references (`&usize`), and you internally modified `robots[last_idx]`, which leads to the contradiction of "simultaneous mutable and immutable borrowing". Please modify through temporarily retrieve the index and separate the mutable reference with scope.

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

Rust Code:

```

rust ◇ 折叠复制

#[derive(Debug)]
struct Robot {
    position: i32,
    health: i32,
    direction: char,
    index: usize,
}

impl Solution {
    pub fn survived_robots_healths(positions: Vec<i32>, healths: Vec<i32>, directions: String) -
        let n = positions.len();
        let directions: Vec<char> = directions.chars().collect();
        let mut robots: Vec<Robot> = (0..n)
            .map(|i| Robot {
                position: positions[i],

```

```

        health: healths[i],
        direction: directions[i],
        index: i,
    ))
    .collect();
robots.sort_by_key(|x| x.position);
let mut stack: Vec<usize> = Vec::new();
let mut result: Vec<Option<i32>> = vec![None; n];
for i in 0..n {
    if robots[i].direction == 'R' {
        stack.push(i);
        continue;
    }
    let mut current_health = robots[i].health;
    let current_index = robots[i].index;
    while let Some(&last_idx) = stack.last() {
        let last_health = robots[last_idx].health;
        let last_direction = robots[last_idx].direction;
        let last_index = robots[last_idx].index;
        if last_direction != 'R' {
            break;
        }
        if last_health == current_health {
            result[last_index] = None;
            result[current_index] = None;
            stack.pop();
            current_health = 0;
            break;
        } else if last_health > current_health {
            robots[last_idx].health -= 1;
            result[current_index] = None;
            current_health = 0;
            break;
        } else {
            result[last_index] = None;
            stack.pop();
            current_health -= 1;
        }
    }
    if current_health > 0 {
        result[current_index] = Some(current_health);
    }
}

```

```
        result[current_index] = Some(current_health);
    }
}
for &idx in &stack {
    result[robots[idx].index] = Some(robots[idx].health);
}
result.into_iter().filter_map(|x| x).collect()
}
}
```

